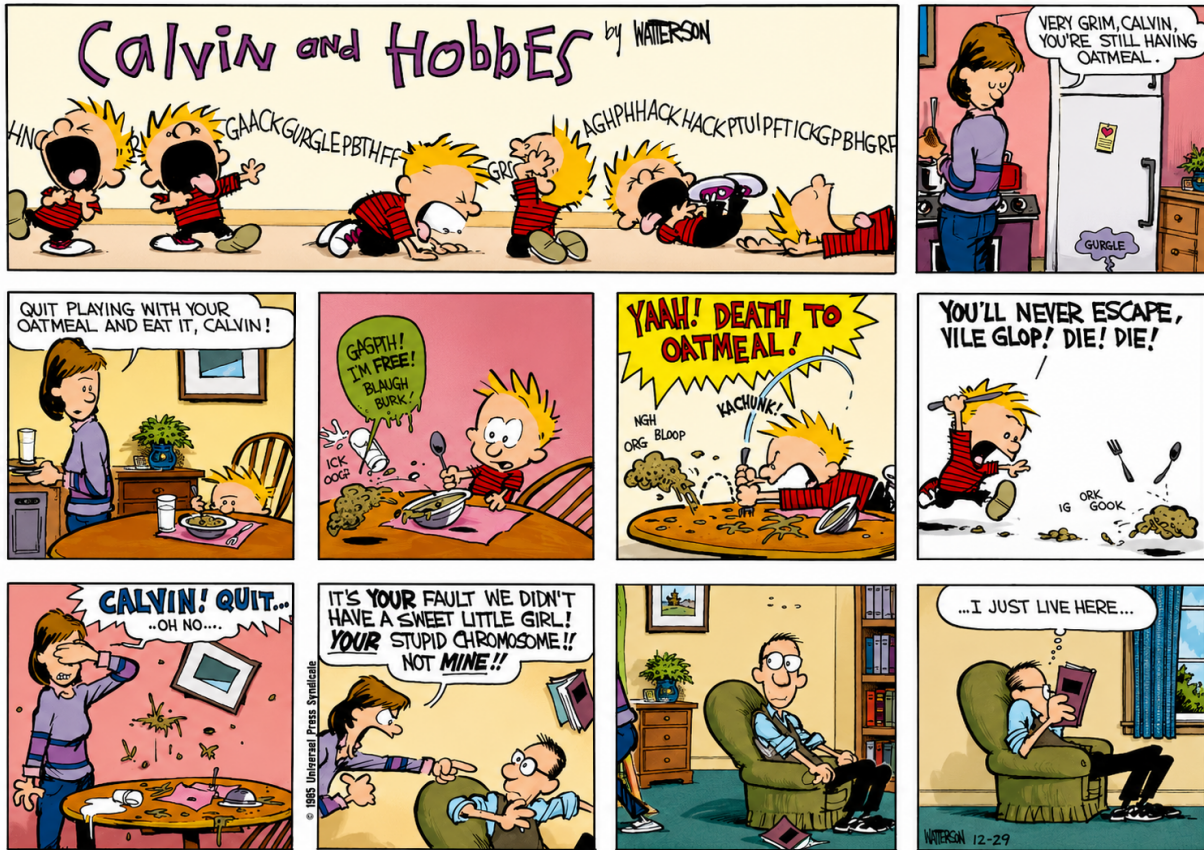
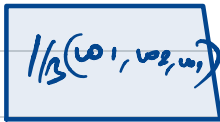




Sensor Parallelism

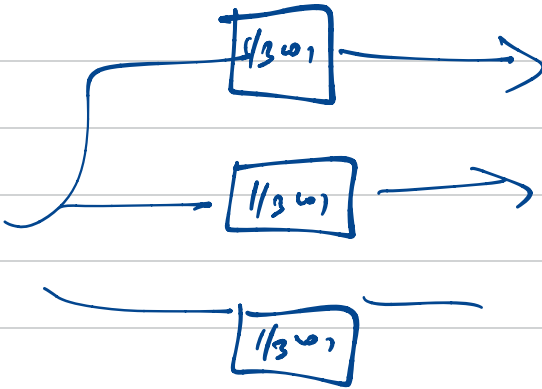
$\omega_1, \omega_2, \omega_3$



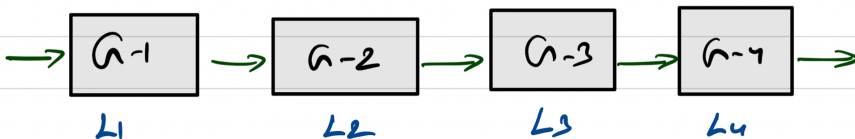
G_1

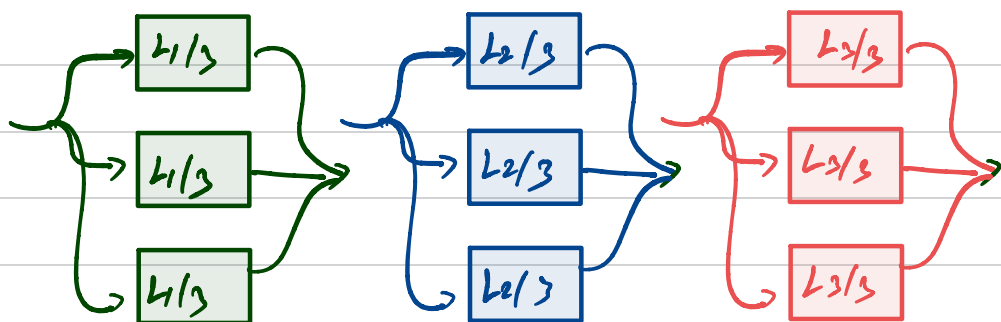
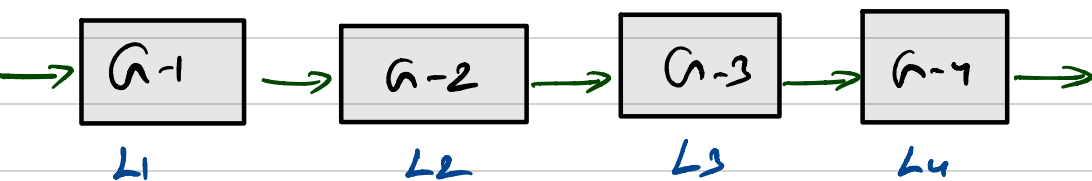


$1/3(\omega_1, \omega_2, \omega_3)$



③ Pipeline Parallelism





Model Quantization

bfloat16

float32: IEEE single-precision



float16: IEEE half-precision



<https://www.youtube.com/watch?v=yvdtwKF87Ts>

3 brts

2



010

$$2^3 - 1$$

0-7

3 - 011

9

bfloat16



$S \rightarrow 0/1$

$Pos(+)\rightarrow 0$

$Neg(-)\rightarrow 1$

float32: IEEE single-precision



float16: IEEE half-precision



A floating point number can be represented by three things:

sign x significant digits x scale

$-372000 = 3.72 \times 10^5$
sign-bit = 1
mantissa *Scale*

19280

$\rightarrow 1.928 \times 10^4$
mantissa *Scale*

$$7.42 \times 10^{-3} = 0.00742$$

$$7.42 \times 10^{-1} = 0.742$$

$$7.42 \times 10^0 = 7.42$$

$$7.42 \times 10^2 = 742$$

$$7.42 \times 10^5 = 742,000$$

The mantissa controls precision

$$4.736281 \times 10^3$$

$$4.736 \times 10^3$$

if I go to lower precision

More mantissa digits

→ more significant digits preserved

→ greater precision

Fewer mantissa digits

→ more rounding

→ lower precision

Precision means retaining nearby differences

Consider these four numbers:

4.721

4.724

4.727

4.729

A high-precision format may preserve all four separately.

A lower-precision format that keeps only two decimal places may store them

as:

4.72

4.72

4.73

4.73

Two different original values now map to the same stored value.

With even lower precision, they might all become:

4.7

bfloat16



in brain float 16, we give more weightage to scale/exponent than mantissa

float32: IEEE single-precision



Float 32 $[-3.4 \times 10^{38}, 3.4 \times 10^{38}]$

Float 16 $[-6.55 \times 10^4, 6.55 \times 10^4]$

BFloat 16 $[-3.39 \times 10^{38}, 3.39 \times 10^{38}]$

float16: IEEE half-precision

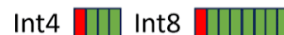


Float 8 $[-240, 240]$

float32 -> float 16 a

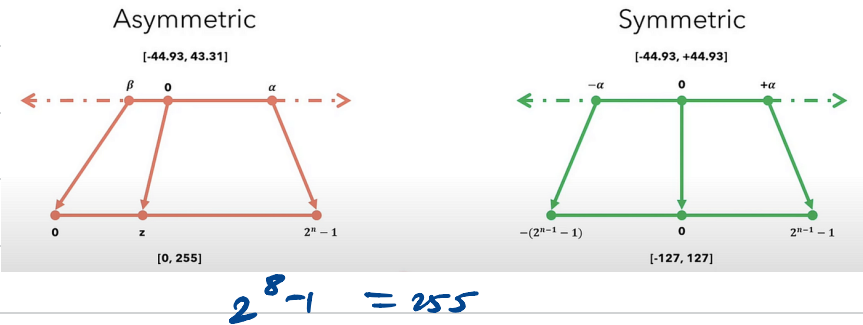
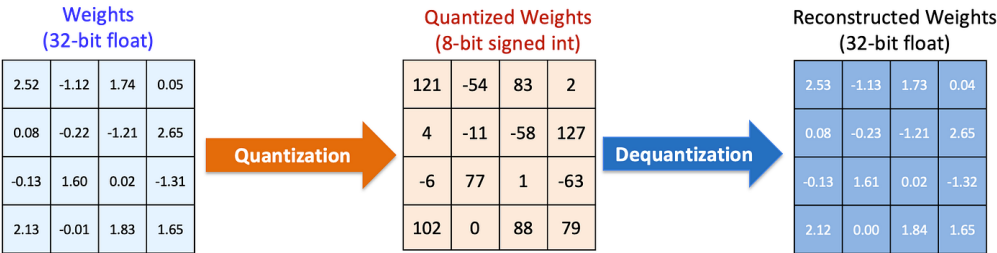
or float32 -> int8 b

Sign Exponent Mantissa



Regime

How Quantizations Work?



<https://pub.towardsai.net/a-brief-practical-guide-to-llm-quantization-b5fafec0d2eb>



Situation	Usually preferred
Values balanced around zero	Symmetric
Values mostly positive	Asymmetric
Values mostly negative	Asymmetric
Neural-network weights	Usually symmetric
ReLU activations	Usually asymmetric or unsigned
Simpler and faster integer computation required	Symmetric
Maximum use of the available integer range required	Asymmetric
Highly skewed distribution	Asymmetric
Approximately zero-centred distribution	Symmetric

Model Variant	Parameters	VRAM (FP16)	VRAM (4-bit Quantization)
DeepSeek-LLM 7B	7 billion	16 GB	4 GB
DeepSeek-LLM 67B	67 billion	154 GB	38 GB
DeepSeek V2 16B	16 billion	37 GB	9 GB
DeepSeek V2 236B	236 billion	543 GB	136 GB
DeepSeek V2.5 236B	236 billion	543 GB	136 GB
DeepSeek V3 671B	671 billion	1,543 GB	386 GB

<https://api.proxpc.com/media/uploads/2025/01/28/vram-deepseek-table-1.png>

①

Quantize Aware Training (QTA)

②

Post Quantize
training

ASYMMETRIC

$$q = \text{round} \left(\frac{x}{\text{scale}} \right) + \text{zero-point}$$

$$\text{Scale} = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}} ;$$

1

zero-point

$$\text{round} \left(q_{\min} - \frac{x_{\min}}{\text{scale}} \right)$$

$$\text{scale} = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$$

$$\text{zero_point} = \text{round} \left(q_{\min} - \frac{x_{\min}}{\text{scale}} \right)$$

$$n = [-1.0, -0.4, 0, 1.2, 3.7, 6.0]$$

$$q_{\min} = 0, \quad q_{\max} = 255$$

$$n_{\min} = -1, \quad n_{\max} = 6$$

$$Scale = \frac{6 - (-1)}{255 - 0} = \frac{7}{255} \approx 0.027$$

$$\text{Zero point} = 36$$

$$q = \text{round} \left(\frac{n}{Scale} \right) + \text{zero-point}$$

$$n_{\text{approx}} = Scale(q - z)$$

```
1 round(-1/(7/255)) + 36
```

✓ Last Execution: 10:14:04 PM, Duration

0

4.23423412343

We have right now 8 billion parameters. I quantize them, to 8 bits, and save scale/zero point for dequantization.

Post training, I quantize and save weights, then I load the weights, dequantize and use it for inference.

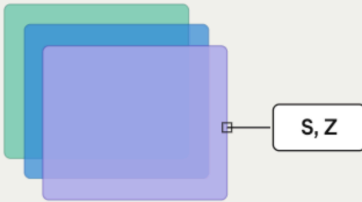
7x7x512

4.74234 -> 4.7

25k -> 256

Per-tensor vs per-channel quantization

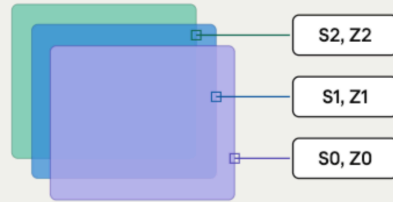
Per-tensor quantization



Every element uses the same mapping:

- Uses minimal metadata
- No extra compute overhead
- Can introduce significant rounding errors

Per-channel quantization



Unique scale and zero-point per channel:

- Captures unique distribution of values per channel
- Reduces quantization noise, preserving accuracy
- Must store and apply one pair of parameters per channel

FP16

$$\left(\frac{n - m}{6} \right)$$

Model → Training done

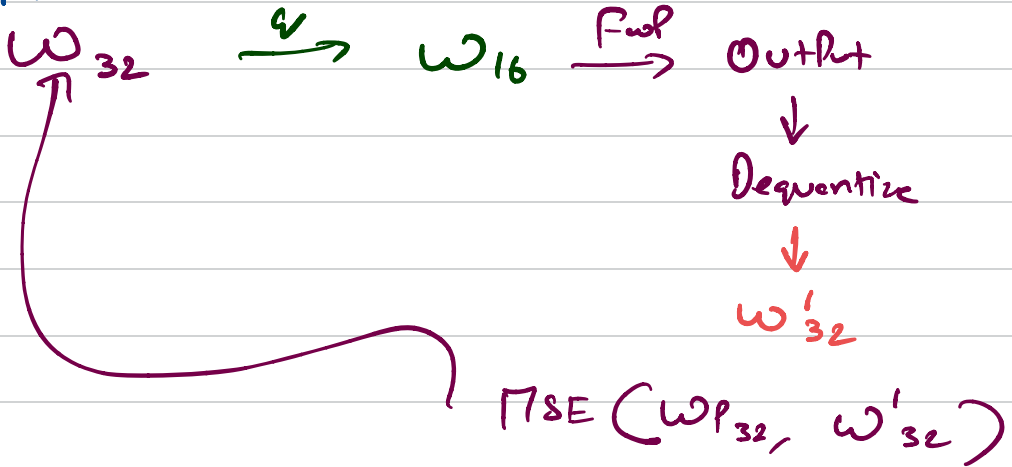
↳ Quantize

↓
smaller wts

↓
Inference ← Dequantize

Mixed Precision Training

master wts



$$4.12342134234 * 4234234.4234234$$

$$4.1234 * 4323.423$$